OurApp_vs_Obsidian

Phase 1 | Finding & organising notes + Editing & reading

Transparent expert UX comparison • 6 November 2026

Main conclusion: Our strongest visible problem is not a shortage of buttons. It is that navigation, editing controls and feedback do not yet form consistently simple task flows. The evidence supports targeted presentation and predictability work; it does not support declaring Obsidian faster in milliseconds.

What this document is: A manual expert review of retained real app screenshots, supported by bounded source inspection. It contains 20 prioritised findings, exact evidence references, consequences, proposed directions and hands-on acceptance checks for two modules.

What it is not: No new live touch session, participant usability study, physical-device performance trace or fresh bug-reproduction run was performed for this report. This is not an E2E pass-count comparison. Live smoothness and completion-time rankings remain unmeasured.

Baseline	Evidence used
Our app	MyNotes 0.69.0 / code 69, retained run 42021523354. Signed APK receipt: 155,195,444 bytes.
Comparator	Official Obsidian Android 1.15.0 / code 396, reviewed native captures from run 42179340885, retained in report 42182162974.
Scope	Shared note retrieval, organisation, writing and reading tasks. No feature-count contest; app-specific extras do not earn points or excuse friction.
Changes	Documentation/report tooling only. No app implementation or Calculator changes; no Android build or install in this review.

Read the evidence type on every finding. A visible issue in a retained frame is not automatically a newly reproduced current-device bug.

Executive judgment: where our app is lagging

1. Content competes with controls N01 and E01 show concrete presentation failures: a create action covers a preview, and formatting with the keyboard consumes writing context.

2. Concept boundaries are weak N03–N06 mix browsing, creating, opening, moving and storage administration. The user has to understand app structure before completing an ordinary note task.

3. Search needs a clear contract N07–N09 identify source-supported risks: a query broadens scope, body matches lack a passage, and filtered-empty can read like first-use empty. These need live verification, not dismissal because a test suite passed.

4. Reading and editing need distinct visual priorities E07–E09 show management chrome and inconsistent saved/location feedback. A feature can exist and still make users hesitate.

5. Long-note orientation needs a better established route Obsidian’s captured outline is a concrete navigation reference. Our reviewed evidence does not establish an equivalent heading-jump path; confirm this candidate gap before designing it.

First triage order — not a promise that fixes are already done

Start with N01 preview obstruction and E01 editor viewport. Then manually reproduce N07 search scope and N09 filtered-empty wording. Next simplify Library/folder roles and mixed-content control scope. Investigate live performance in parallel; do not assume visual cleanup alone removes jank.

Do not replace weak evidence with a numerical UX score

There is no defensible overall score, success percentage or speed winner here. The fixtures, Android environments, input methods and capture timings were not matched. Prior automation outcomes remain correctness receipts, not usability ratings.

Method, evidence strength and fairness rules

Evidence label	Meaning and limit
Visible defect/constraint	Personally inspected in a retained screenshot. The captured presentation can be described; recurrence on the current signed build needs reproduction.
Source-supported candidate	A specific current-checkout method establishes a branch or renderer design. It is not an observed live execution of that scenario.
Expert inference	A reasoned usability concern based on visible hierarchy, feedback or mental model. Validate with real tasks and users.
Unknown / not established	No matched live result or sufficient evidence. No assumption that a missing screenshot means a missing feature.

The 33 source frames were visually inspected for this review. Whole frames are scaled proportionally and not cropped to hide controls, recoloured or reconstructed. Image source IDs, paths, dimensions and SHA-256 hashes appear in the accompanying evidence manifest.

MyNotes and Obsidian images use different fixture content and retained JPEG sizes; apparent sharpness is not a typography-quality measurement. Their screen image aspect ratios do not establish identical physical devices, DPI, IME or OS. Portrait comparisons are the primary scope. Obsidian’s attempted landscape remained portrait in its audit; that is not a valid landscape benchmark against our app.

Source inspection baseline: bf39d35dc625a86a84f44d03a178da4569a2b594. Main MyNotes references: NotesController.renderHome, filterNow, renderUniversalResults, renderFolders, renderEditor and FormatControls. Source inspection is distinguished from screenshot evidence on each finding.

The Obsidian audit explicitly did not achieve 100% functional acceptance. Empty search and blocked action captures are not used as proof of superior search relevance, reliability or smoothness. A visual reference is useful without pretending the competitor is flawless.

Your five observations: supported, unproven and next checks

Your observation	Transparent assessment in these two modules
Existing features need polish	Supported by visible obstruction, clipped wording, panel crowding and competing content controls. Details: N01/N02/E01/E07. Not evidence every feature is defective.
Many features are missing	Candidate navigation/action-discovery gaps are noted, but no blanket absence claim. Existing global search, folders, labels, grid/list and shared formatting are acknowledged. Other modules are deferred.
Pages do not feel simple	Supported as an expert judgment: menu roles, storage-heavy folder browsing, layered move/create and mixed-object chrome increase interpretation work.
150+ MB but feels empty	The APK receipt is 155.20 decimal MB; Obsidian's verified APK was 14.51 MB, about 10.7× smaller. That is APK payload, not installed footprint/RAM/speed. Sparse test libraries do not establish a real-user empty home. The issue to investigate is time-to-first-useful-outcome and discoverability, not filling space with fake content.
Many bugs	Your report remains valid input. This inspection does not establish a current crash count or reproduce every bug. N01 is a visible frame defect; N09 is a source-supported reproduction candidate. Neither a green suite nor this report closes your field issues.

No APK contents breakdown was performed, so model/runtime packaging is not asserted as the cause of size, jank or bugs. Nor is an extra-feature list used to excuse weaknesses in common note workflows.

Comparison dimensions: what can actually be concluded

Dimension	Phase-one judgment
Presentation / UI hierarchy	Assessable in reviewed states: our home obstruction, tall formatting surface, mixed-object chrome and menu concepts need work.
Simplicity / learning	Expert inference: our action grouping is inconsistent; Obsidian provides clearer task-specific surfaces but has icon and Markdown learning costs.
Efficiency	Extra scanning and lost context are plausible. Actual taps, time, corrections and recovery costs have not been measured head-to-head.
Effectiveness	Selected states and source behaviour can be examined. Matched task completion, data correctness and real-user success still need live evaluation.
Smoothness / responsiveness	Not ranked. No matched frame-time, jank, touch-to-feedback, ANR or startup recordings were collected. Recognition timing is not editor-scroll timing.
Reliability / save confidence	Save-feedback presentation differs. No current data-loss or comparative crash-rate conclusion is justified.
Accessibility / ergonomics	Labelled controls are a useful starting point. Target sizes, TalkBack, contrast and large-text behaviour require direct validation; no invented WCAG scores.
Value / completeness	Judge complete shared tasks. APK size and automation counts do not measure how useful or finished the app feels.

Finding index and triage legend

P1 = address or validate early because it affects common task confidence, content visibility or orientation. P2 = polish/learning improvement or a hypothesis needing frequency evidence. Priority is an expert triage recommendation, not measured incidence.

Find & organise

ID / priority	Finding
N01 / P1	The floating create action covers preview text
N02 / P2	The search invitation is clipped
N03 / P1	Library mixes destinations with creation and intake
N04 / P1	Notes and Doc ask for an early classification decision
N05 / P2	Folder browsing carries storage-management weight
N06 / P2	Creating a folder adds a second sheet over Move to folder
N07 / P1	Global queries silently change the filtering contract
N08 / P1	Universal results do not explain a body-text match
N09 / P1	A filtered-empty library can look like a new empty library
N10 / P2	A broad search is not yet a clear action-discovery route

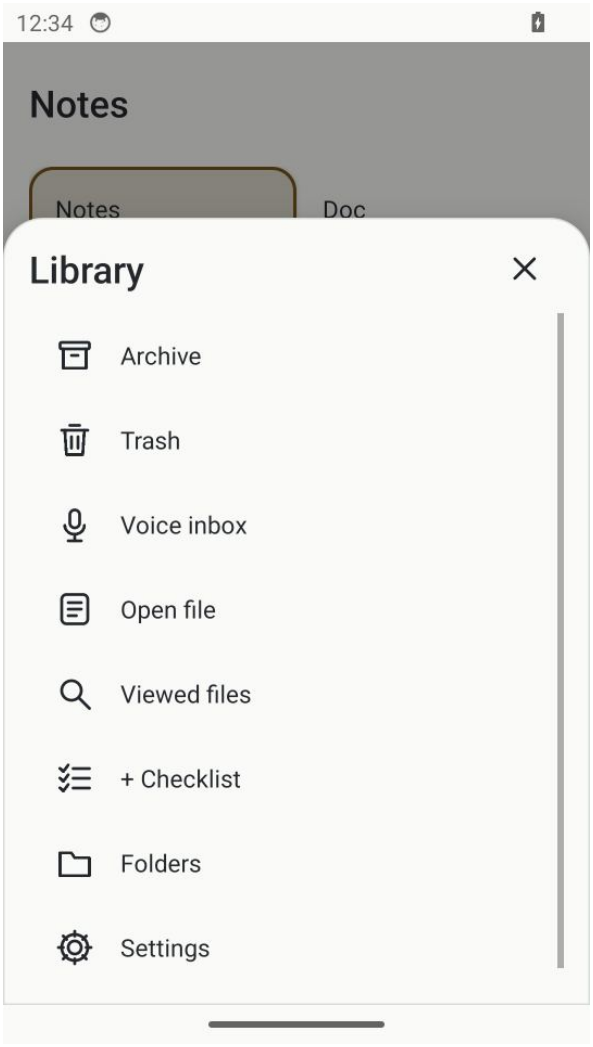
Edit & read

ID / priority	Finding
E01 / P1	Formatting plus keyboard squeezes out the writing context
E02 / P1	Selection versus future typing is not explicit enough visually
E03 / P2	Routine formatting shares space with advanced controls
E04 / P2	Text-size controls expose implementation units
E05 / P2	The persistent editor toolbar gives decoration equal weight
E06 / P1	Long notes lack an established heading-jump counterpart
E07 / P1	Mixed content looks like editable machinery even while reading
E08 / P1	Several nearby menus require users to infer their scope
E09 / P1	Save/location confidence differs between note surfaces
E10 / P2	Find-result context uses implementation-oriented wording

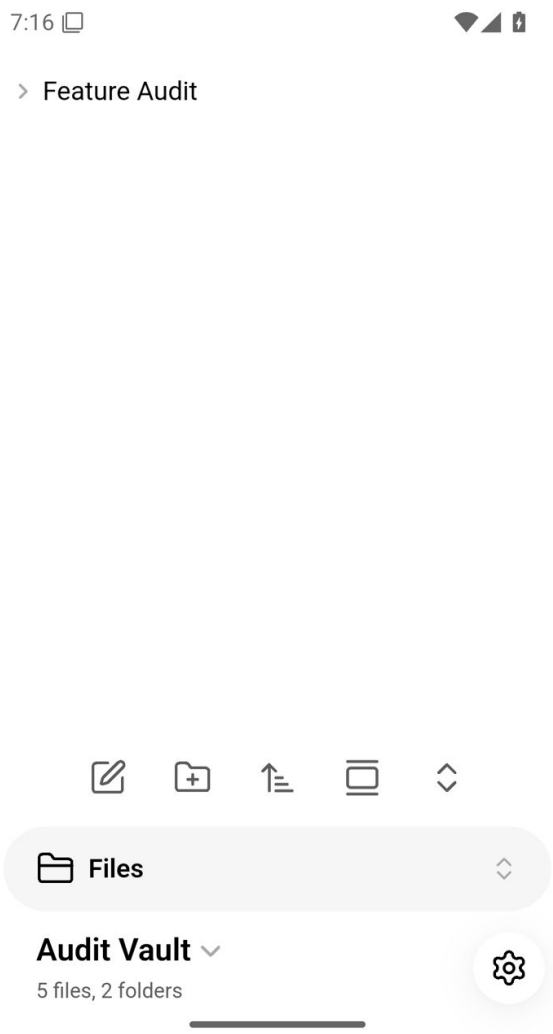
Module: Find & organise

Shared user goal: Find existing content, understand where it lives, organise it and return without relearning storage or workspace rules.

This module has ten scoped findings. They do not represent ten confirmed runtime bugs or a measured competitor pass/fail score.



M02 | MyNotes 0.69.0 / code 69 | library



O02 | Obsidian 1.15.0 / code 396 | File explorer, fixture folder collapsed

Different navigation models, not matched record sets: Library action/destination menu versus a file-tree pane with its fixture folder collapsed. Compare conceptual grouping; do not infer directory depth or fewer taps from these pictures.

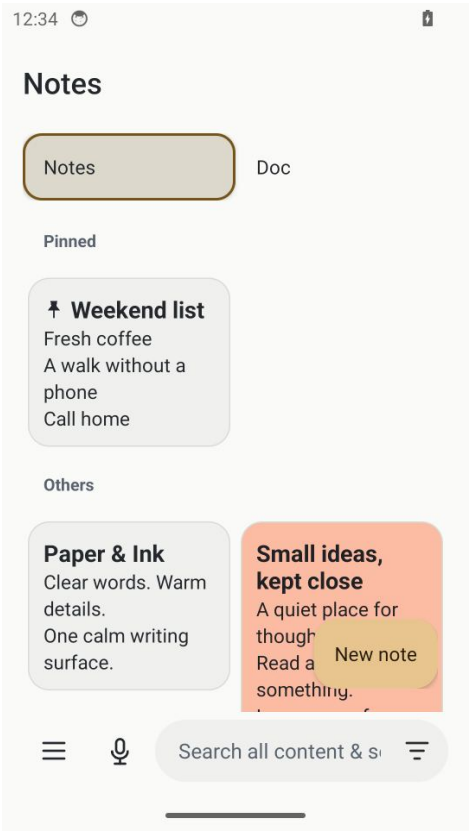
N01 | The floating create action covers preview text

P1 • Visible defect in retained frame

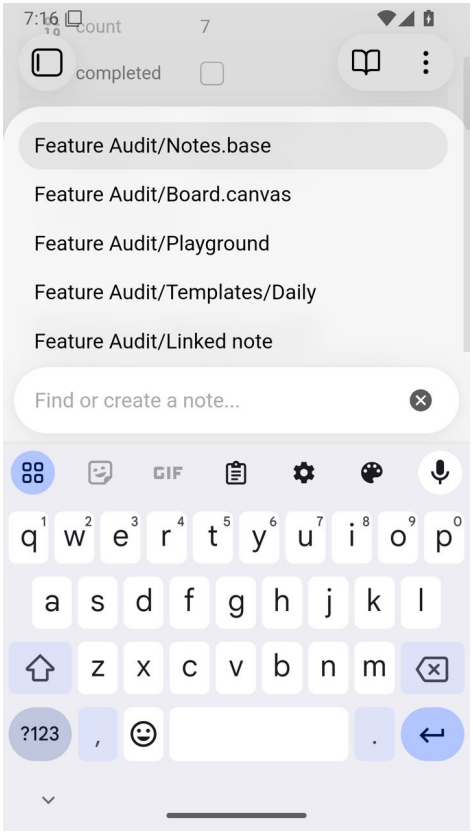
User task: Read the last visible note preview, then create a note.

Evidence: In M01, New note sits over words in the orange note card. This is a visible obstruction, not just an aesthetic preference.

Comparator: O01 keeps file-choice rows and its search input distinct. It is a different retrieval surface, not proof that every Obsidian overlay avoids content.



M01 | MyNotes 0.69.0 / code 69 | home



O01 | Obsidian 1.15.0 / code 396 | Quick switcher with file paths and Find or create input

Why it matters: The same small region must support reading and tapping; confidence in the card preview is reduced.

Direction to consider: Keep the requested floating New note button. Reserve a readable safe area or adapt its placement; do not restore the rejected bottom create strip.

Manual acceptance: Recreate the three-card state, then test long titles, last-row scroll, list/grid, keyboard and large text. No required preview line or row action should be covered at rest.

Source anchor: NotesController.renderHome: floating createRow; board bottom padding. Limit: Not newly reproduced on a device. This does not prove the card is impossible to open or scroll.

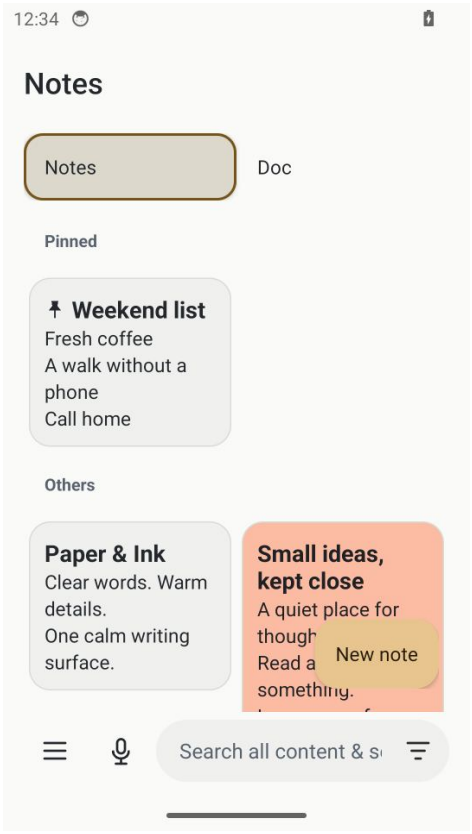
N02 | The search invitation is clipped

P2 • Visible presentation limitation

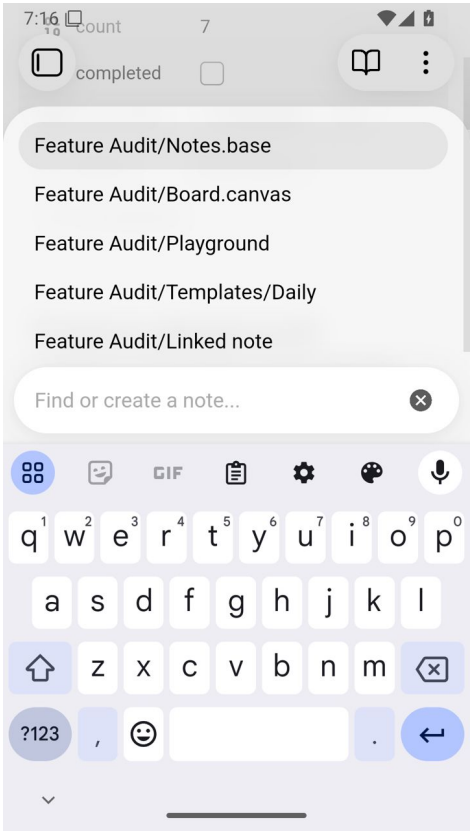
User task: Understand what the home search will search.

Evidence: M01 shows the hint clipped after Search all content & s.... The full source hint is Search all content & settings.

Comparator: O01 uses Find or create a note; O03 presents a separate vault-search surface. Obsidian also has icon-learning costs.



M01 | MyNotes 0.69.0 / code 69 | home



O01 | Obsidian 1.15.0 / code 396 | Quick switcher with file paths and Find or create input

Why it matters: The promise of a broad search is hidden exactly where a first-time user needs scope clarity.

Direction to consider: Use a shorter visible invitation and an explicit scope explanation in the expanded search state, within the existing single bottom search.

Manual acceptance: At small width and large text, the primary search invitation remains understandable without guessing the clipped suffix.

Source anchor: NotesController.renderHome: search.setHint and fixed filter control. Limit: Clipping is visible; comprehension impact remains an expert judgment, not a user-study result.

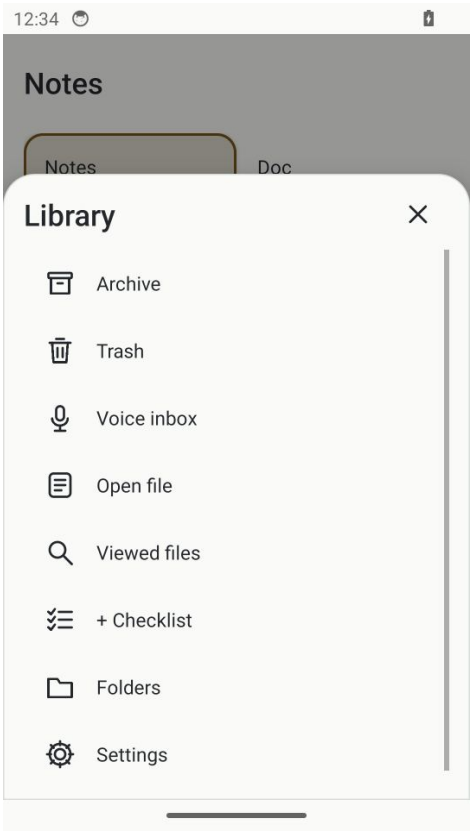
N03 | Library mixes destinations with creation and intake

P1 • Expert UX inference from visible UI

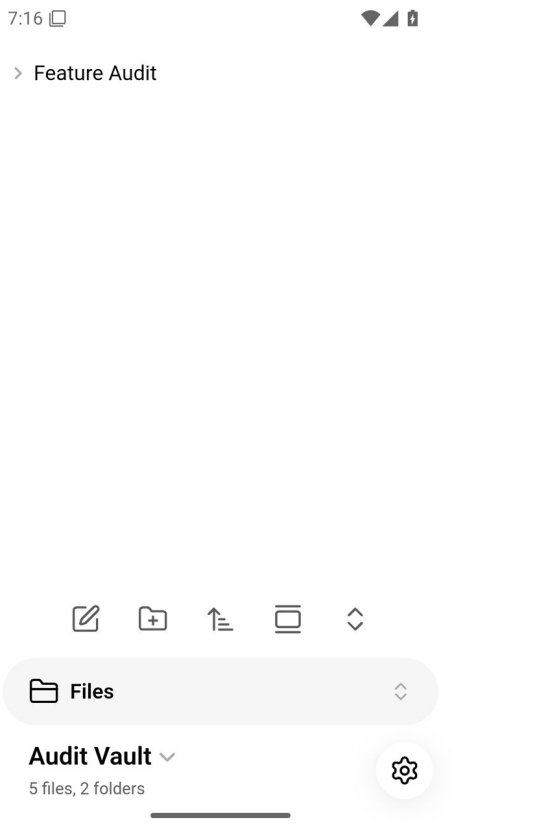
User task: Choose between browsing existing work, creating content and managing the app.

Evidence: M02 mixes Archive, Trash, Voice inbox, Open file, Viewed files, + Checklist, Folders and Settings in one undivided list.

Comparator: O02 is a file-browsing surface; O04 separately exposes actions. This is a concept-boundary comparison, not equivalent menu inventories.



M02 | MyNotes 0.69.0 / code 69 | library



O02 | Obsidian 1.15.0 / code 396 | File explorer, fixture folder collapsed

Why it matters: Users must infer which entries open places and which start operations. The app can feel like a collection of tools rather than one workspace.

Direction to consider: Group the existing menu by purpose and make creation/intake roles consistent. Retain requested Folders/Settings ordering and do not add duplicate Notes/Doc entries.

Manual acceptance: Ask new users where to find a saved note versus create a checklist; record wrong turns before and after grouping.

Source anchor: NotesController.renderHome: navigation/menuRows. Limit: No measured wrong-turn rate is available. Obsidian also distributes actions across several surfaces.

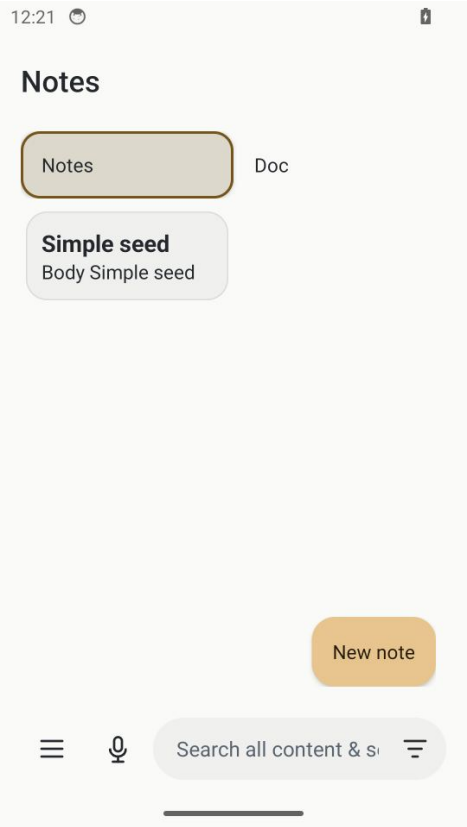
N04 | Notes and Doc ask for an early classification decision

P1 • Navigation hypothesis requiring hands-on confirmation

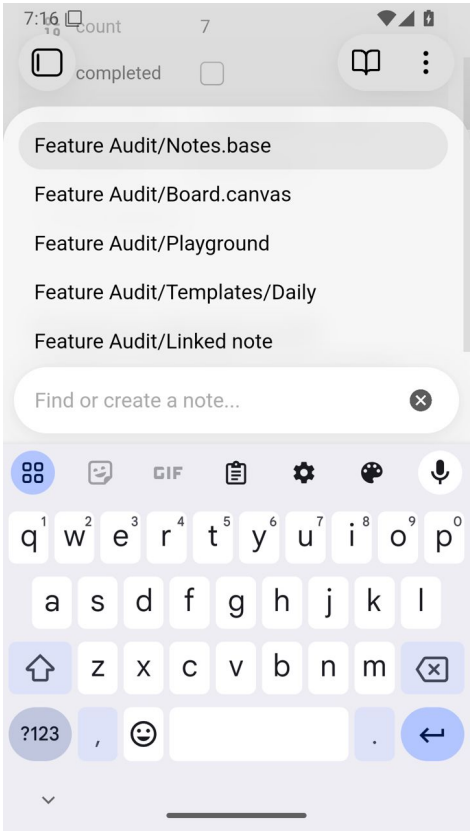
User task: Retrieve yesterday's content without remembering its creation mode.

Evidence: M01 and M12 prominently separate Notes and Doc before showing content. Source implements independent workspace filtering.

Comparator: O01 presents several file kinds in one chooser. That does not mean MyNotes lacks cross-workspace search: its universal search already crosses Notes and Docs.



M12 | MyNotes 0.69.0 / code 69 | notes-workspace



O01 | Obsidian 1.15.0 / code 396 | Quick switcher with file paths and Find or create input

Why it matters: Mode recall can become an extra burden when users remember subject matter, not whether something was a Note or Doc.

Direction to consider: Preserve both data models. Make retrieval scope and the current workspace obvious; use the existing global search as a reliable bridge rather than duplicating home controls.

Manual acceptance: Use mixed Notes/Docs with similar titles. Compare successful retrieval, workspace switches and backtracking, without instructing the creation mode.

Source anchor: NotesController.filterNow; Workspace.includes; renderUniversalResults. Limit: Not a claim that tabs are intrinsically bad or that the two models must be merged.

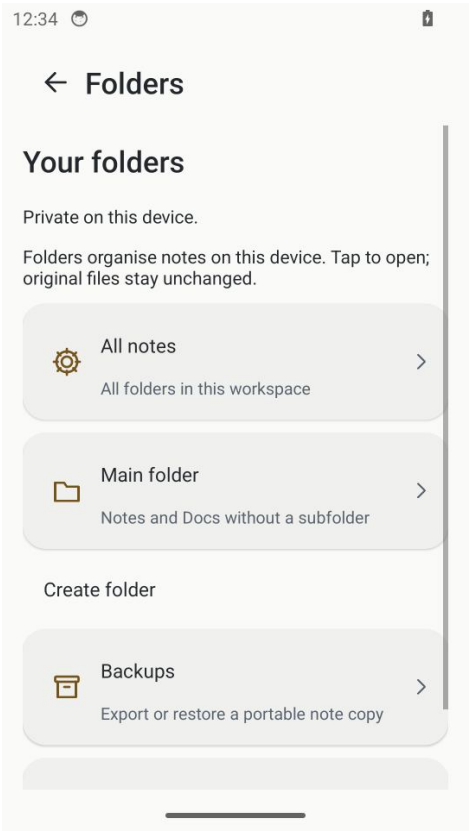
N05 | Folder browsing carries storage-management weight

P2 • Visible structure plus UX inference

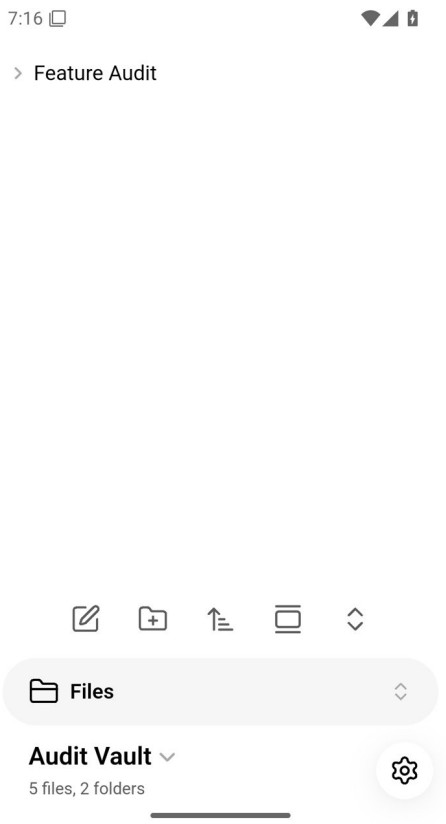
User task: Open a folder during ordinary note organisation.

Evidence: M03 repeats Folders / Your folders, explains private storage and originals, then includes Backups; source also appends Storage settings.

Comparator: O02 focuses on a collapsed fixture-folder tree. We did not execute a matched folder-navigation task in both apps.



M03 | MyNotes 0.69.0 / code 69 | folders



O02 | Obsidian 1.15.0 / code 396 | File explorer, fixture folder collapsed

Why it matters: Safety explanations are useful, but they compete with the everyday hierarchy and contribute to a settings-page feeling.

Direction to consider: Keep accurate storage disclosures available, but progressively disclose management details away from the main folder list. Do not remove recovery routes.

Manual acceptance: With ten folders, the main task is immediately recognisable as browsing. Users can still locate backup and storage details without ambiguity.

Source anchor: NotesController.renderFolders. Limit: The sparse fixture is not evidence that real users have no folders.

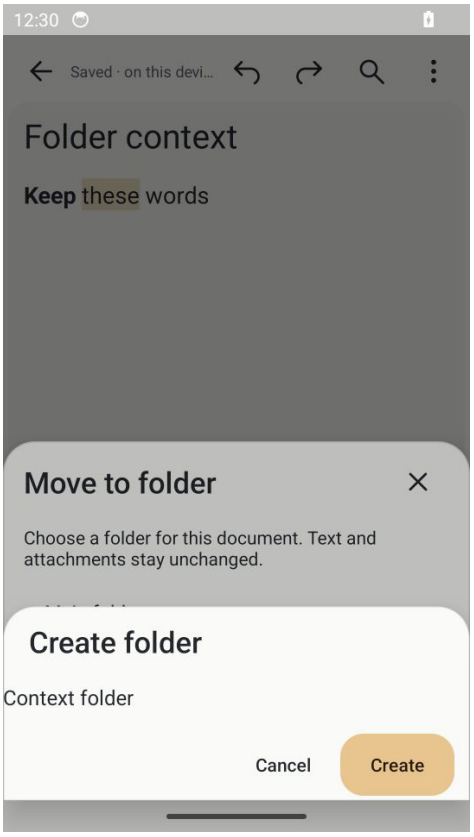
N06 | Creating a folder adds a second sheet over Move to folder

P2 • Visible layering; behaviour unmeasured

User task: Create a destination while moving the current item.

Evidence: M11 contains the dimmed editor, Move to folder and a second Create folder surface. Multiple dismiss/return levels are visible.

Comparator: No matched Obsidian create-while-moving capture is available. This is a directly observed MyNotes issue, not a claimed competitor speed win.



No matched Obsidian interaction capture is available for this specific flow. This is not a competitor speed or success-rate comparison.

M11 | MyNotes 0.69.0 / code 69 | editor-folder-create

Comparator: not established

Why it matters: Users have to predict whether Create also moves the item, which layer closes, and where Back returns.

Direction to consider: Use one clear staged flow inside the existing move surface, retaining the note and returning to the chosen destination. Keep the actual move outcome explicit.

Manual acceptance: Exercise Create, Cancel, system Back, invalid names and keyboard dismissal. Verify the correct destination and unchanged note contents after each branch.

Source anchor: NotesController.renderMoveFolderSheet and renderFolders. Limit: No claim of data loss or broken Back behaviour is established by this image.

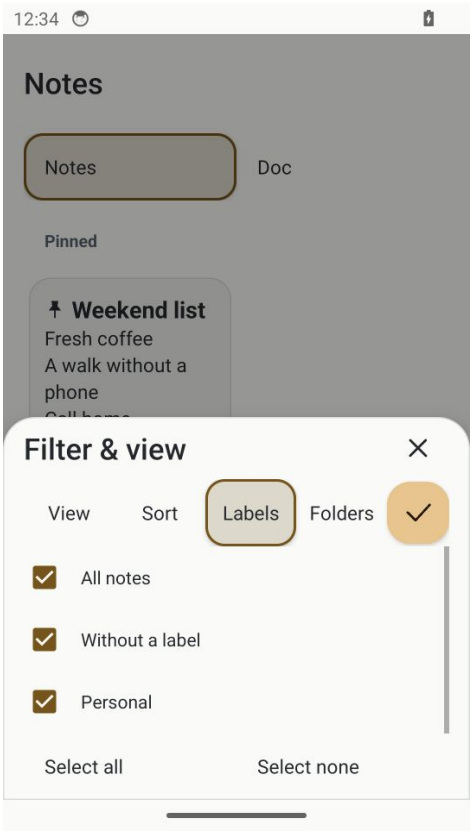
N07 | Global queries silently change the filtering contract

P1 • Source-supported scope mismatch risk

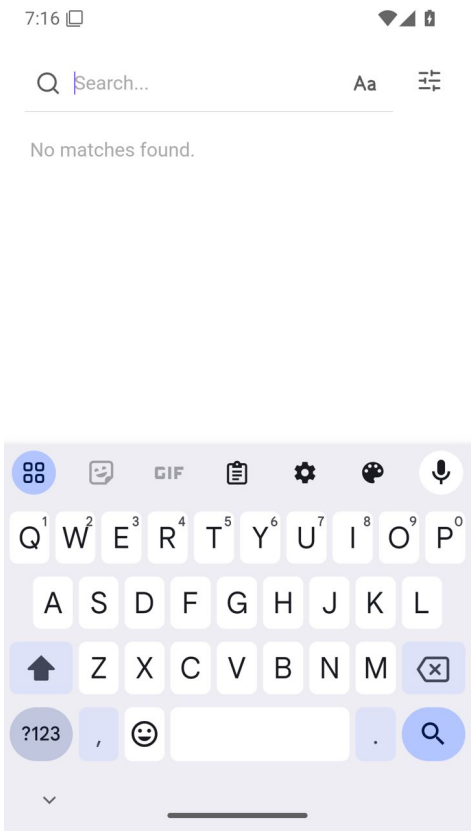
User task: Search while a folder or label filter is active.

Evidence: filterNow makes every nonempty query on page Notes global: folder, workspace and label filters are bypassed; nondeleted archived notes are included. This is deliberate code behaviour, not a measured bug.

Comparator: Obsidian separates quick switching, vault search and tags in O01/O03/O05. Their combined filtering semantics were not tested.



M05 | MyNotes 0.69.0 / code 69 | filters-labels



O03 | Obsidian 1.15.0 / code 396 | Vault search, empty query; not search-result quality evidence

Why it matters: A user can think the query narrows the current list while the implementation broadens it. That undermines predictability.

Direction to consider: Keep global search if intended. State All notes & Docs / includes archived explicitly and make switching to current-scope search understandable.

Manual acceptance: Populate two folders, two labels, Notes/Docs and an archive item. For each scope/query combination, verify the displayed scope and returned IDs against the stated contract.

Source anchor: NotesController.filterNow: global; included; global || filter expression; renderUniversalResults. Limit: Source establishes the branch. Its actual presentation with a populated global query needs fresh live capture.

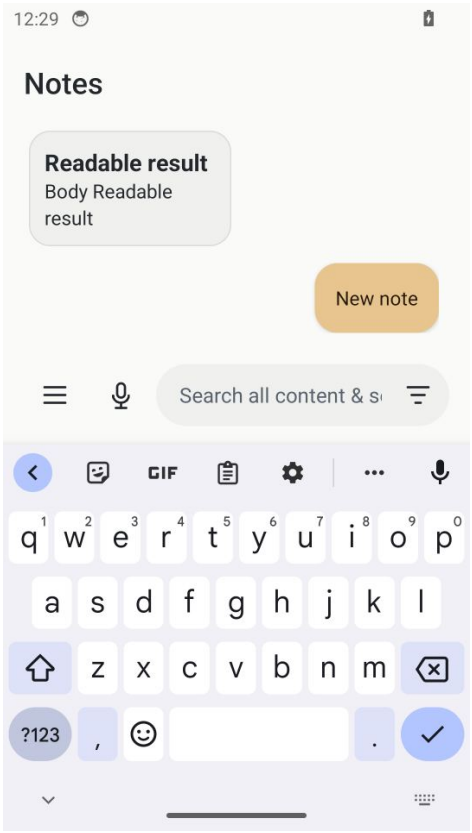
N08 | Universal results do not explain a body-text match

P1 • Source-supported presentation gap

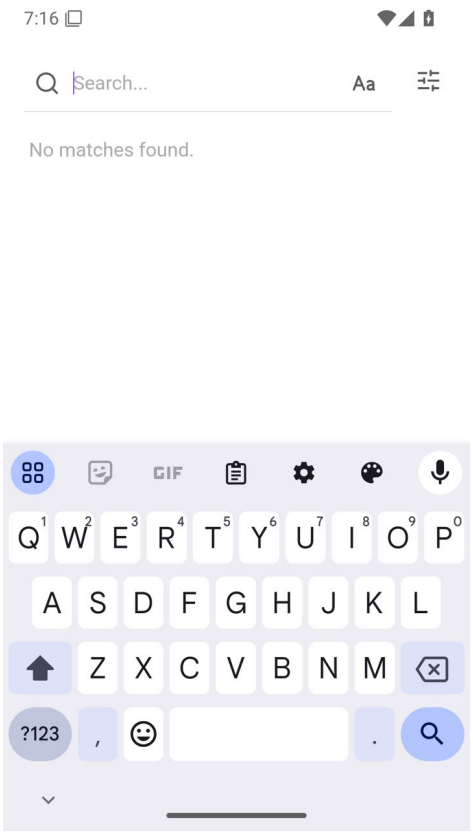
User task: Identify the right note when the search phrase is only in its body.

Evidence: renderUniversalResults builds title buttons plus Note/Doc, archive and folder metadata. That renderer does not add a matching body passage or query highlight.

Comparator: O03 is an empty-query capture, so no claim about superior Obsidian snippets or search accuracy is made here.



M06 | MyNotes 0.69.0 / code 69 | search-keyboard



O03 | Obsidian 1.15.0 / code 396 | Vault search, empty query; not search-result quality evidence

Why it matters: Similar titles require opening candidates to discover why they matched; finding information is not the same as merely returning an item.

Direction to consider: Add a compact, safe matching passage with highlighted terms and retain the already useful type/folder metadata. Preserve original text and bound long snippets.

Manual acceptance: Create five identically titled items with different body phrases. Identify the intended item from the result context, then verify the correct open target.

Source anchor: NotesController.renderUniversalResults: title button and type/folder line. Limit: M06 shows a focused search entry, not a matched global query. Source evidence supports this finding; comparative relevance is unmeasured.

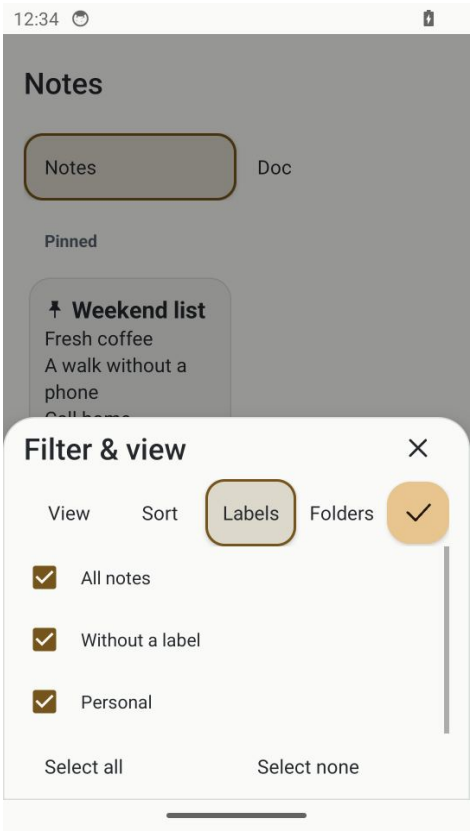
N09 | A filtered-empty library can look like a new empty library

P1 • Source-supported defect candidate; not live reproduced

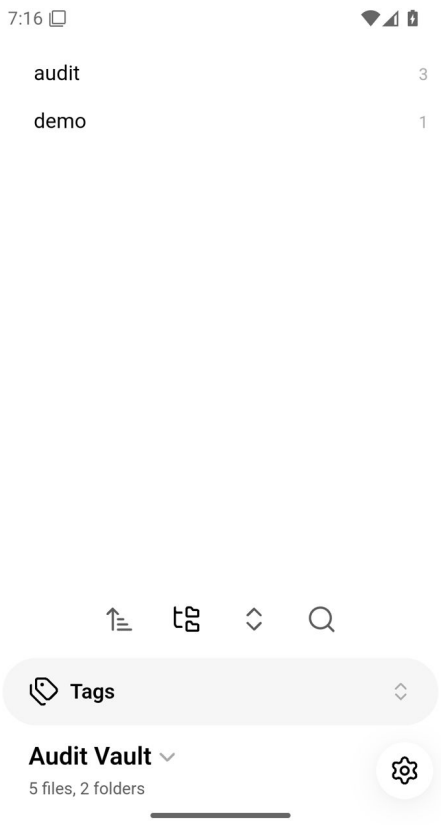
User task: Understand why existing notes are not visible.

Evidence: The emptyHint expression has a special no-labels-selected branch, but an ordinary nonempty label filter with zero matches and an empty query falls through to A little space for your ideas / Create a note below.

Comparator: O05 provides a tag-specific browsing context and counts. Its empty-tag state was not captured.



M05 | MyNotes 0.69.0 / code 69 | filters-labels



O05 | Obsidian 1.15.0 / code 396 | Tags pane with usage counts

Why it matters: Users may create duplicate content or worry that saved notes disappeared when only the current view is empty.

Direction to consider: Distinguish first-use empty, no results in this folder/filter, and loading/error states. Offer Clear filters without erasing actual content or preferences unexpectedly.

Manual acceptance: Keep a Work note, select a Personal-only filter, and clear the query. Confirm an accurate filtered-empty explanation and reversible clear action.

Source anchor: NotesController.renderHome: emptyHint ternary; filterNow. Limit: This follows a source branch. Reproduce on the signed build before assigning a confirmed runtime defect.

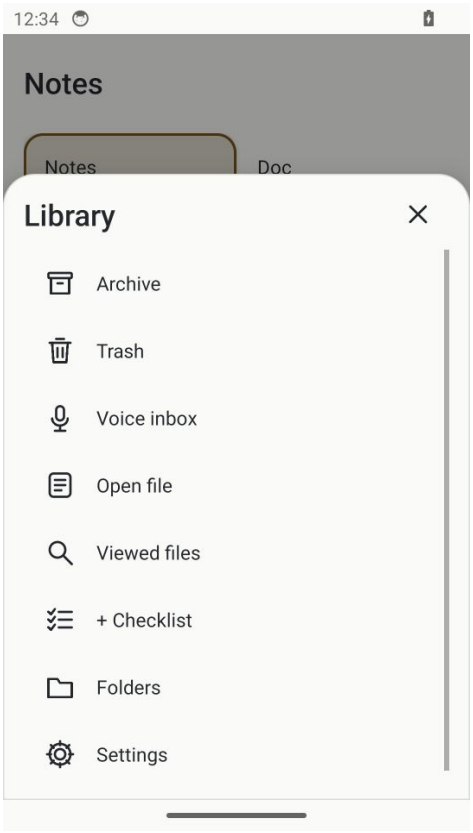
N10 | A broad search is not yet a clear action-discovery route

P2 • Action-discovery hypothesis

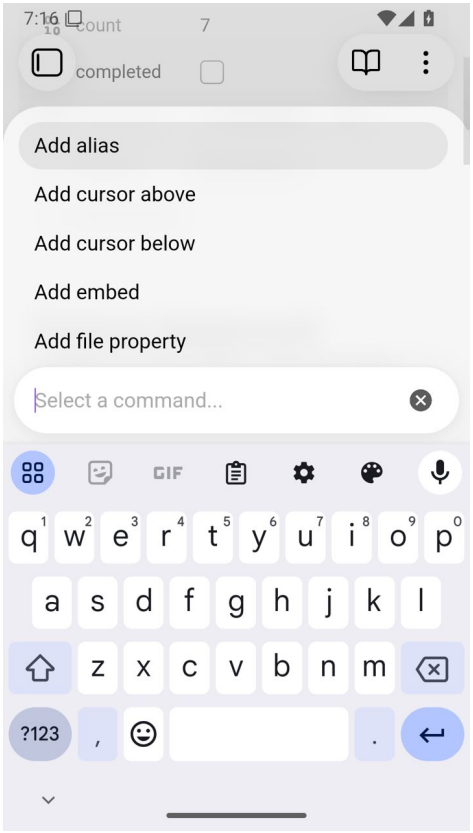
User task: Find a less-frequent action without knowing its menu location.

Evidence: Our Library and editor divide operations across menus and panels. Universal search source includes a small fixed set of settings destinations, not all editing actions.

Comparator: O04 visibly lists commands with a dedicated Select a command input. Execution of every action is not certified by that screenshot.



M02 | MyNotes 0.69.0 / code 69 | library



O04 | Obsidian 1.15.0 / code 396 | Command palette with actions

Why it matters: New users may know what they want to do but not the correct menu, especially after moving between Notes and Doc.

Direction to consider: First improve grouping and wording. If testing still exposes discovery failures, add lightweight action help within existing navigation; do not copy a 205-command interface wholesale.

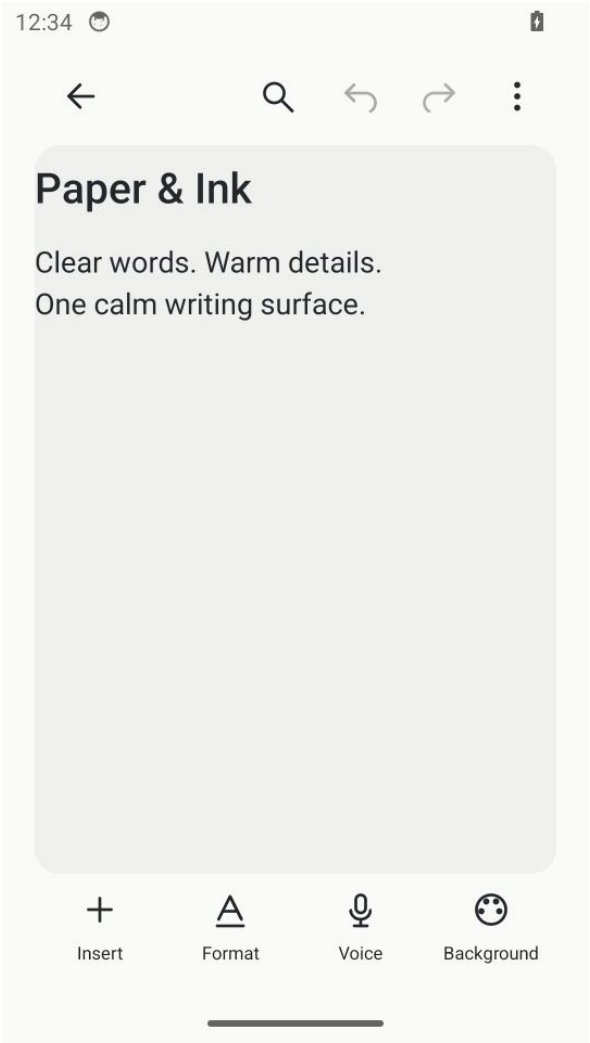
Manual acceptance: Give unprompted tasks such as change heading, move note and find within note. Record menu detours and help requests, not just eventual completion.

Source anchor: NotesController.renderUniversalResults: fixed settings array; renderEditor menus. Limit: No measured task-time advantage or missing-basic-search claim is made.

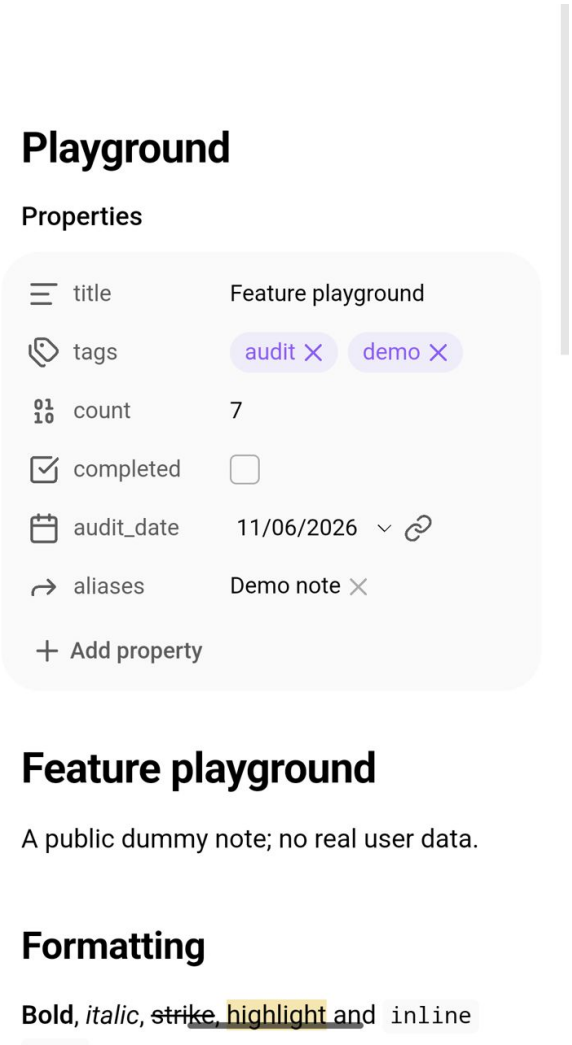
Module: Edit & read

Shared user goal: Write and format accurately, read without distraction, find a section and leave knowing the note is safe.

This module has ten scoped findings. They do not represent ten confirmed runtime bugs or a measured competitor pass/fail score.



M07 | MyNotes 0.69.0 / code 69 | editor



O11 | Obsidian 1.15.0 / code 396 | Properties in note; metadata occupies substantial space

Our plain editor is already calmer in this state; Obsidian's visible properties occupy substantial space before the prose. The goal is not to replace our editor with its metadata-heavy layout.

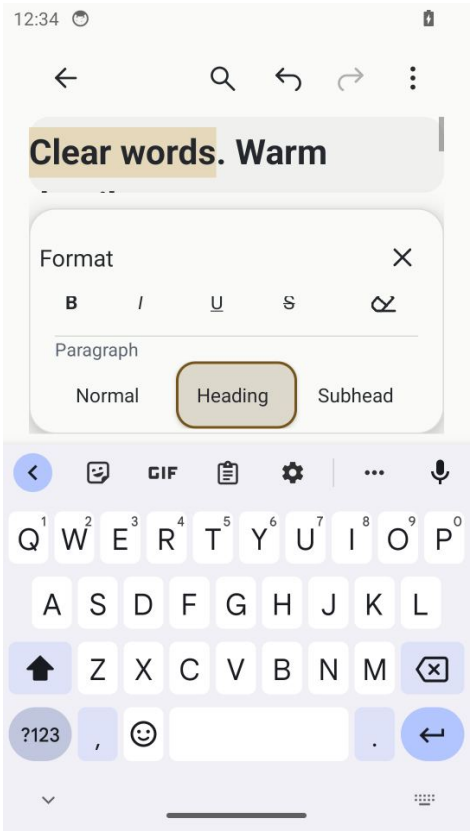
E01 | Formatting plus keyboard squeezes out the writing context

P1 • Visible viewport constraint

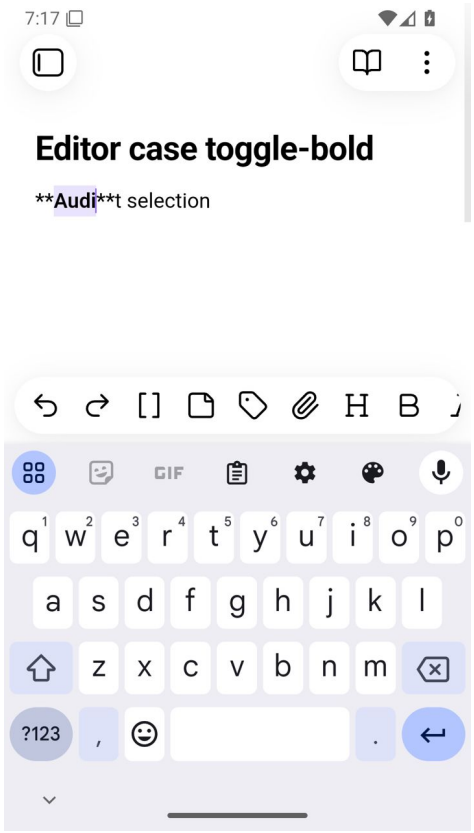
User task: Format a phrase while retaining the surrounding paragraph.

Evidence: M09 leaves only a short band of the note above the Format panel and keyboard. This directly shows how little prose remains visible in that captured state.

Comparator: O06 uses a compact toolbar above the keyboard while more of the note remains visible. Different fixtures prevent a precise productivity measurement.



M09 | MyNotes 0.69.0 / code 69 | format-keyboard



O06 | Obsidian 1.15.0 / code 396 | Bold action on dedicated scratch note; keyboard visible

Why it matters: Users may lose the sentence context, then need repeated panel closing and reopening to continue writing.

Direction to consider: Adapt the existing single Format panel to available height; keep common controls compact and advanced options reachable. Preserve readable labels and the requested single A entry.

Manual acceptance: On the same device/font/IME, format the middle of a paragraph. Target at least three readable context lines at normal font, with a sensible large-text fallback and reachable caret.

Source anchor: NotesController.installFormatting: formattingPanel; FormatControls; adaptEditorViewport. Limit: The proposed context-line target is an acceptance proposal, not a measured current value for all devices.

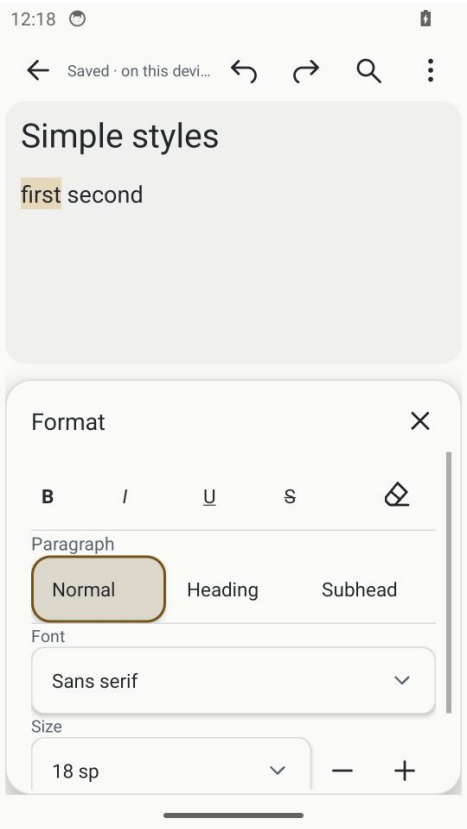
E02 | Selection versus future typing is not explicit enough visually

P1 • Expert UX inference; existing accessible states acknowledged

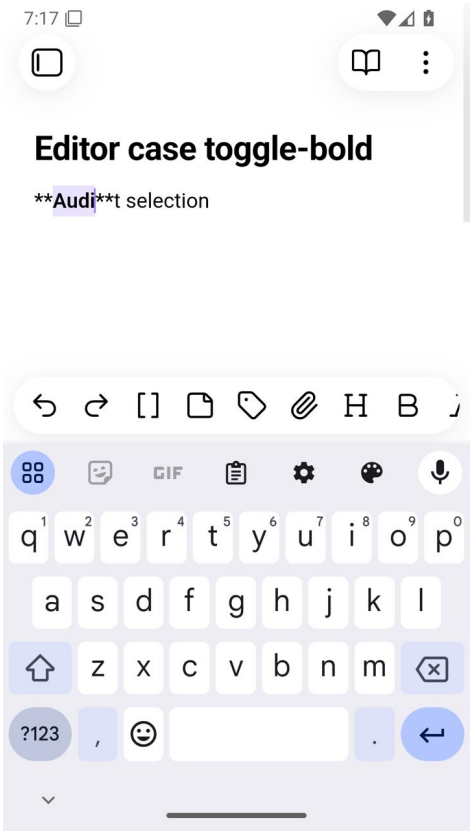
User task: Know whether Bold will affect selected words or the next typed words.

Evidence: M17 and M18 show similar Format surfaces with different selection/caret states. Source has accessibility state descriptions, but the visible panel heading is simply Format.

Comparator: O06 shows an active selection near a compact toolbar. It does not establish that Obsidian solves every mixed-style state.



M17 | MyNotes 0.69.0 / code 69 | stage-aa-selection



O06 | Obsidian 1.15.0 / code 396 | Bold action on dedicated scratch note; keyboard visible

Why it matters: A styling operation can feel unpredictable even when the underlying span logic is correct.

Direction to consider: Clarify the target inside the existing panel heading or concise context text. Preserve current selection, typing state, accessibility descriptions and Clear as the fifth inline action.

Manual acceptance: Alternate selected text, collapsed caret and mixed formatting. Users should predict the target before tapping and observe only the intended text change.

Source anchor: FormatControls.refresh; NotesController.noteTypingStyle and formatNoteValue. Limit: This is a comprehension risk, not a claim that formatting currently edits the wrong range.

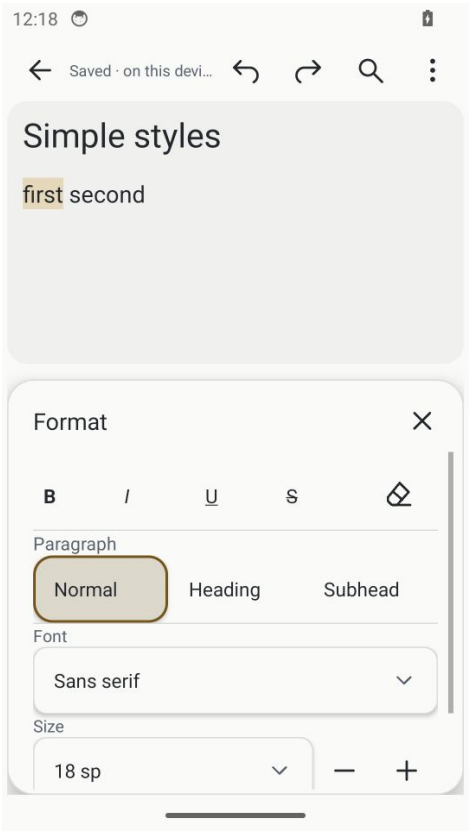
E03 | Routine formatting shares space with advanced controls

P2 • Visible density plus source-confirmed panel contents

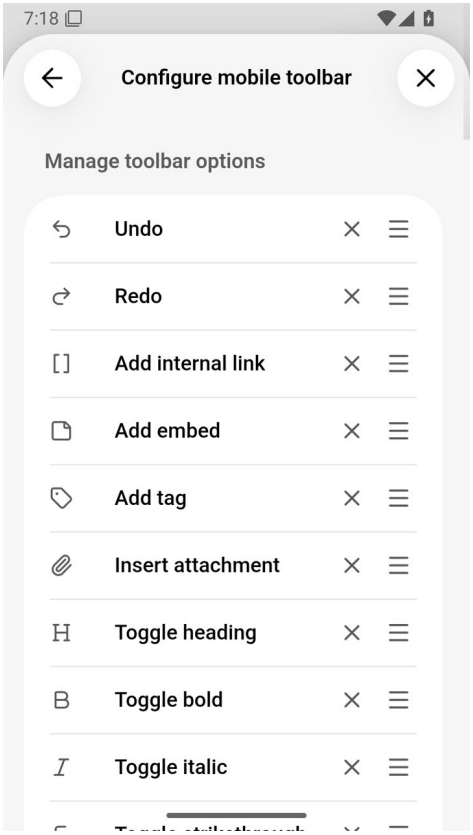
User task: Apply a common style without navigating typography settings.

Evidence: M10/M17 place paragraph choices, font family, Android size values and other options in a tall panel. FormatControls also provides Number paragraphs further down.

Comparator: O10 demonstrates configurable mobile toolbar actions. Customisation is not proof of a better default for novices.



M17 | MyNotes 0.69.0 / code 69 | stage-aa-selection



O10 | Obsidian 1.15.0 / code 396 | Mobile toolbar configuration

Why it matters: More controls do not automatically make editing more effective; frequent actions compete with less-frequent configuration.

Direction to consider: Prioritise frequent actions within the same panel and progressively disclose advanced options. Do not add another formatting strip or remove existing capabilities.

Manual acceptance: Observe five common formatting tasks and one advanced task. The common tasks should not require scanning advanced controls; the advanced task must remain discoverable.

Source anchor: FormatControls constructor and panel ordering. Limit: Action frequency is not measured yet. Validate the prioritisation with actual users.

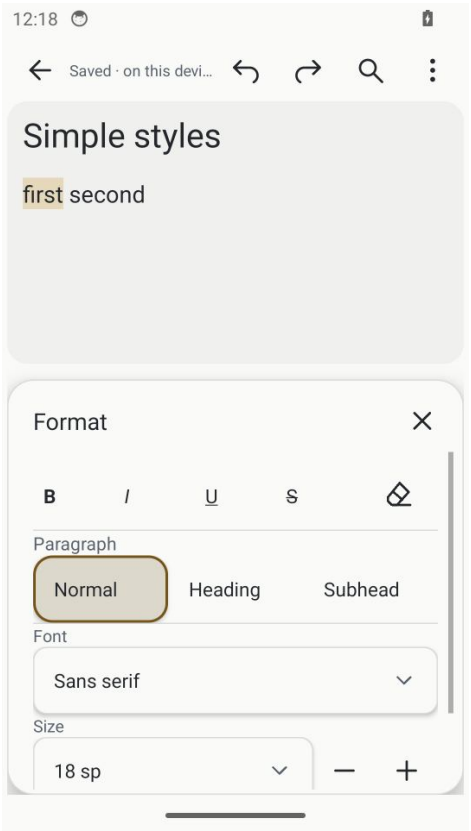
E04 | Text-size controls expose implementation units

P2 • Visible terminology issue

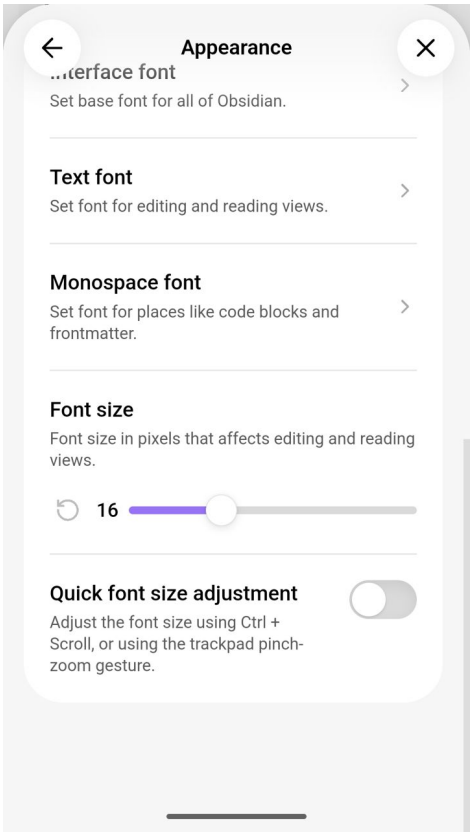
User task: Choose a comfortable reading/writing size.

Evidence: M17 shows 18 sp. This is an Android implementation unit rather than a familiar task description for most note-taking users.

Comparator: O13 also exposes pixels in Appearance. Obsidian is not a plain-language ideal here; its font-size configuration is a different workflow, not numerical equivalence or a proved competitor advantage.



M17 | MyNotes 0.69.0 / code 69 | stage-aa-selection



O13 | Obsidian 1.15.0 / code 396 | Appearance font-size setting

Why it matters: The control adds translation work for users who just want larger or smaller text.

Direction to consider: Use understandable size wording and preview the effect while retaining precise accessible adjustment and the existing allowed range.

Manual acceptance: Ask users to enlarge text one step and restore it without explaining sp. Verify large-text layout, mixed selections and saved sizes remain correct.

Source anchor: FormatControls: sizeChoice, Size (8–72 sp), smaller/bigger. Limit: Terminology judgment only; no unsupported contrast, physical-size or accessibility-failure claim.

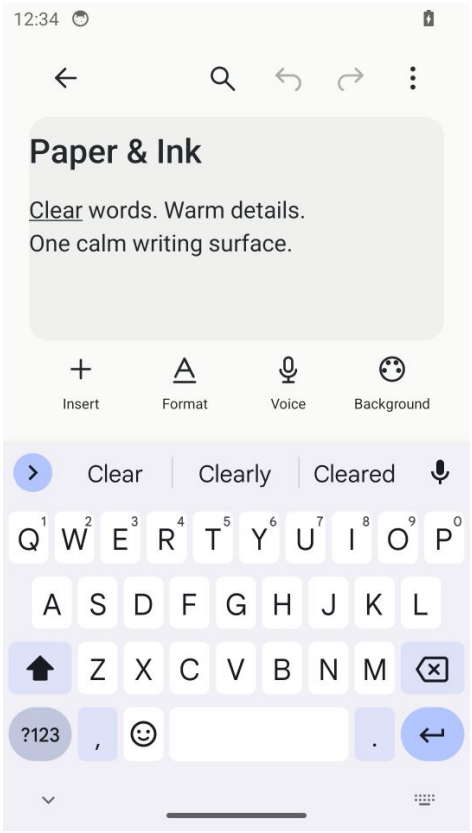
E05 | The persistent editor toolbar gives decoration equal weight

P2 • Priority hypothesis from visible UI

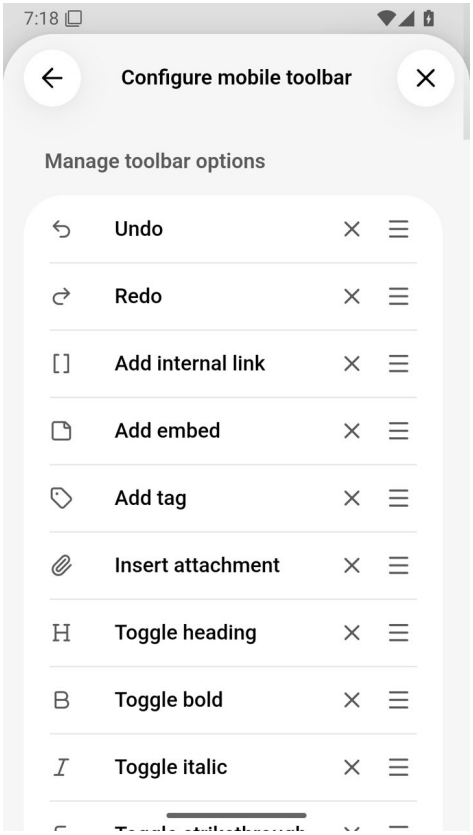
User task: Keep the main writing actions immediately understandable.

Evidence: M07/M08 show Insert, Format, Voice and Background as equal persistent destinations. Background is always at the same hierarchy as writing operations.

Comparator: O10 allows toolbar configuration, but its icon density also increases learning effort. We should not imitate that density indiscriminately.



M08 | MyNotes 0.69.0 / code 69 | editor-keyboard



O10 | Obsidian 1.15.0 / code 396 | Mobile toolbar configuration

Why it matters: For a typing-centred workflow, attention may be spent on decoration while structural actions remain inside other menus.

Direction to consider: Keep the useful labels and required actions. Test contextual prominence or configurable order inside the existing toolbar instead of simply adding more buttons.

Manual acceptance: Run typing, checklist and reorganisation tasks. Determine whether the toolbar order matches user intent before changing defaults.

Source anchor: NotesController.renderEditor writing toolbar. Limit: Voice may be a primary workflow for some users. This is not an argument to remove or devalue it.

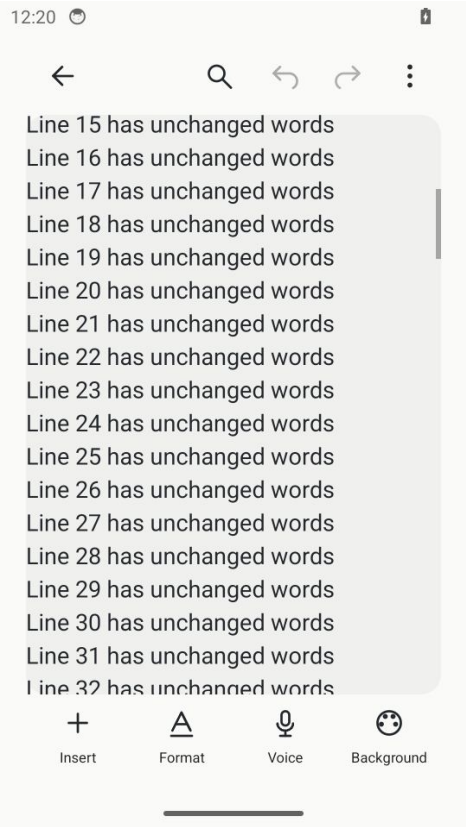
E06 | Long notes lack an established heading-jump counterpart

P1 • Candidate navigation gap in reviewed scope

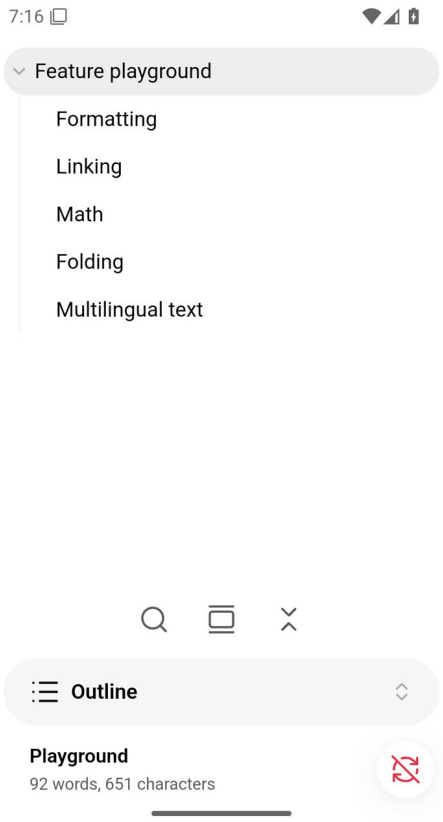
User task: Return to a named section in a long note.

Evidence: M19 demonstrates a long scrollable note. No heading-outline surface was established in the reviewed MyNotes captures; source inspection did not establish a heading navigator.

Comparator: O09 visibly lists the note heading hierarchy in an Outline pane. Exact jump latency and every tap outcome were not measured.



M19 | MyNotes 0.69.0 / code 69 | notes-native-selection-scroll



O09 | Obsidian 1.15.0 / code 396 | Heading outline pane

Why it matters: Scrolling and text search are weak substitutes when users remember document structure rather than an exact phrase.

Direction to consider: Evaluate a lightweight heading navigator that reuses existing heading formatting. Preserve scroll position and make return-to-edit straightforward; avoid introducing graph-style complexity.

Manual acceptance: Use a note with twenty headings and repeated words. Find a named section, edit it, and return to the previous location without losing selection or text.

Source anchor: Reviewed NotesController and runtime Java navigation surfaces; no absence claim beyond this scope. Limit: Potential missing interaction, not proof that all versions or hidden paths lack it.

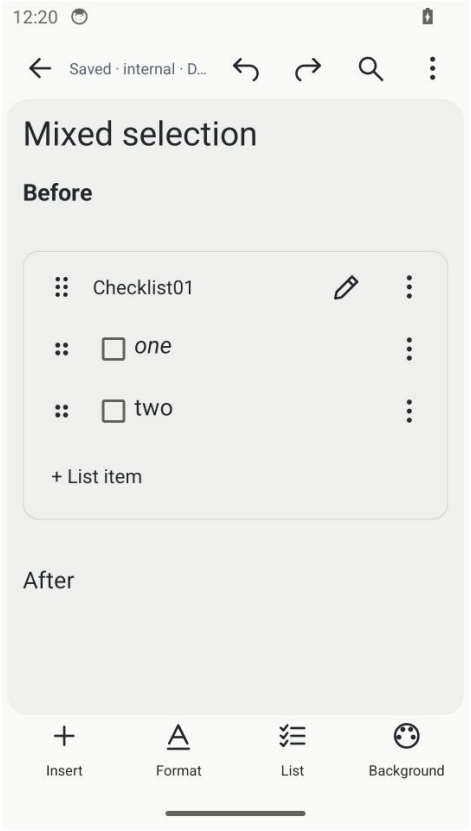
E07 | Mixed content looks like editable machinery even while reading

P1 • Visible presentation gap

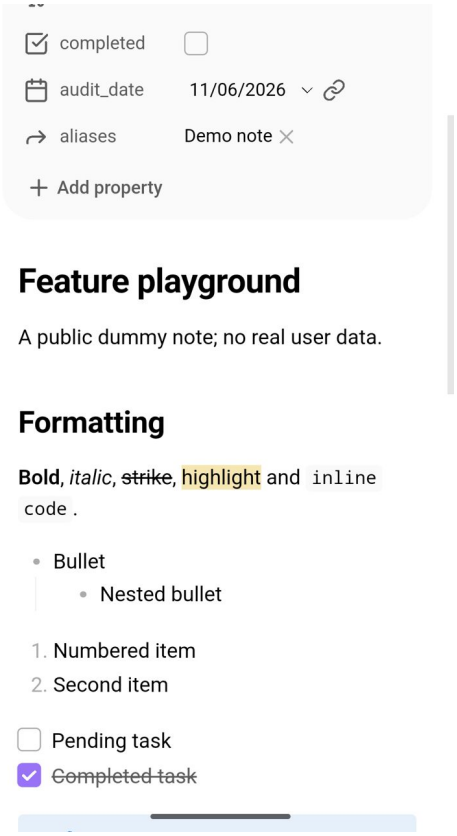
User task: Read prose and a checklist as one coherent note.

Evidence: M13 exposes block borders, a block header, grip, pencil/menu controls and per-row controls between Before and After. The fixture name Checklist01 is test data, not proven default product copy.

Comparator: O07 integrates rendered headings, lists and callouts into note content. Obsidian metadata and its own toolbar still consume space.



M13 | MyNotes 0.69.0 / code 69 | mixed-notes-selection



O07 | Obsidian 1.15.0 / code 396 | Rendered reading view of fixture

Why it matters: Management chrome interrupts the reading hierarchy and can make a small amount of content feel more complicated than it is.

Direction to consider: Offer calmer nonselected presentation and reveal object actions contextually without losing direct editing, grip behaviour, pen-scoped actions or accessibility.

Manual acceptance: Read a mixed note without editing, then edit one row and reorder one block. Compare comprehension and accidental edits while ensuring every existing action is still reachable.

Source anchor: NotesController.renderEditor and content block renderers. Limit: This does not justify merging all object types or removing the existing safety controls.

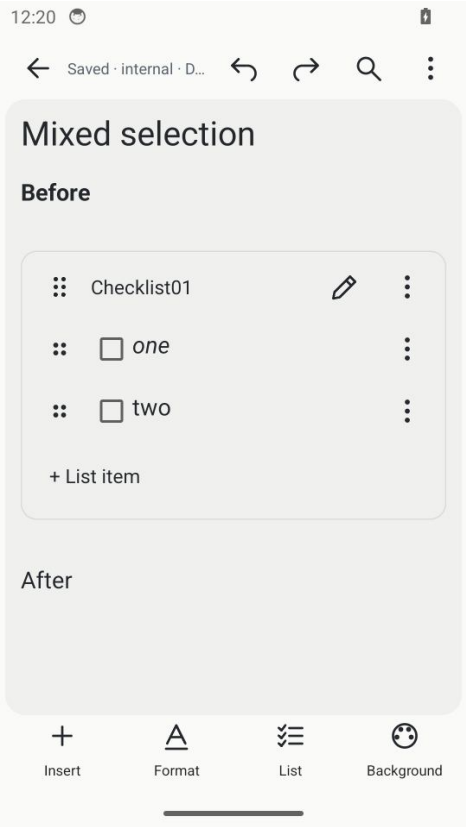
E08 | Several nearby menus require users to infer their scope

P1 • Control-scope comprehension risk

User task: Rename a block versus change or delete an individual list item.

Evidence: M13 has a block-level pencil/overflow and multiple row overflows. The visual relationship between each action and its object must be learned.

Comparator: O08 illustrates integrated reading, not a verified equivalent list-management menu. No competitor interaction advantage is assumed.



No matched Obsidian interaction capture is available for this specific flow. This is not a competitor speed or success-rate comparison.

M13 | MyNotes 0.69.0 / code 69 | mixed-notes-selection

Comparator: not established

Why it matters: Choosing the wrong scope risks unwanted structural changes and increases hesitation around editing.

Direction to consider: Make selected-object boundaries and action wording explicit while retaining per-content actions, per-cell operations and confirmation before permanent deletion.

Manual acceptance: Ask users to change one row, rename its block and remove only that row. Record wrong-menu openings and verify unaffected content remains byte/semantically unchanged.

Source anchor: NotesController content headers and item action menus. Limit: Existing content descriptions are not assumed missing; TalkBack interaction was not tested in this review.

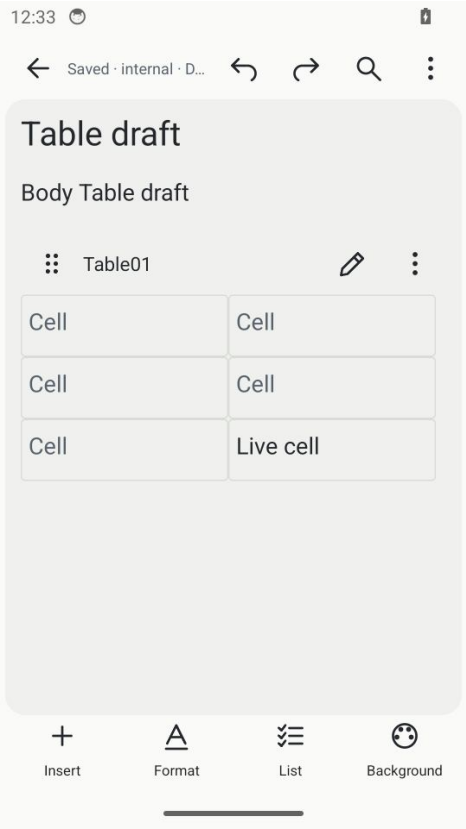
E09 | Save/location confidence differs between note surfaces

P1 • Visible feedback inconsistency

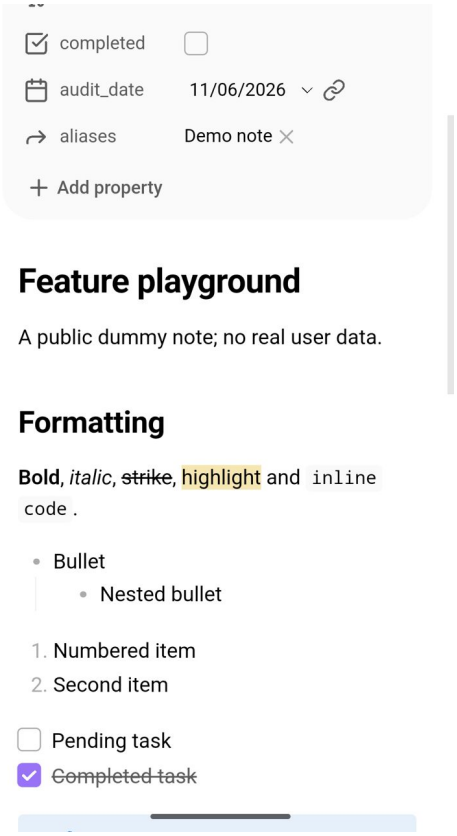
User task: Leave the editor knowing where the work is and whether it is saved.

Evidence: M07 has no visible saved-status text in its captured top bar. M14/M11 show a status string truncated to Saved · internal · D... or Saved · on this devi....

Comparator: Obsidian exposes a file/path context on several views, but we did not inspect a comparable failure/recovery indicator. It is not a proven save-reliability winner.



M14 | MyNotes 0.69.0 / code 69 | document-table



O07 | Obsidian 1.15.0 / code 396 | Rendered reading view of fixture

Why it matters: The user has to infer safety differently across surfaces; truncation removes meaning from already compact status text.

Direction to consider: Define concise shared saving/saved/error feedback and a readable location detail on demand. Keep one compact top row; do not add permanent duplicate status bars.

Manual acceptance: Edit, wait for save, leave, reopen and inject a controlled write failure on disposable data. Status must match persisted content and keep recovery/Back safe.

Source anchor: NotesController.renderEditor; document header/status surfaces. Limit: No lost save or data corruption was reproduced. This is feedback design, not a persistence-failure assertion.

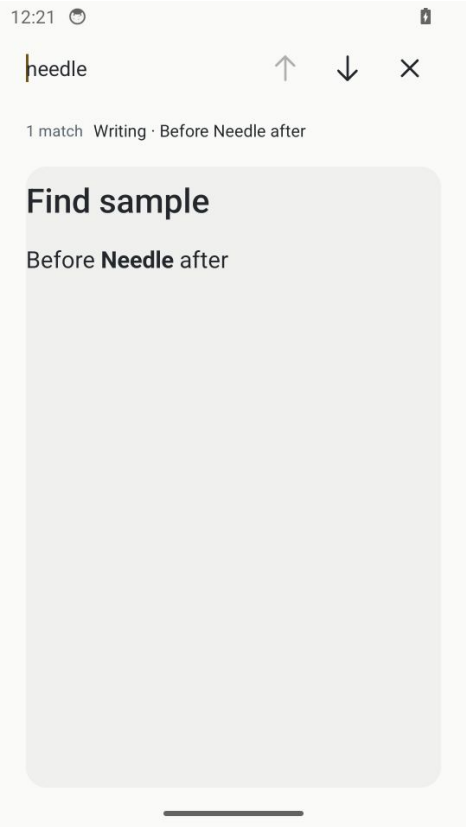
E10 | Find-result context uses implementation-oriented wording

P2 • Visible copy/presentation concern

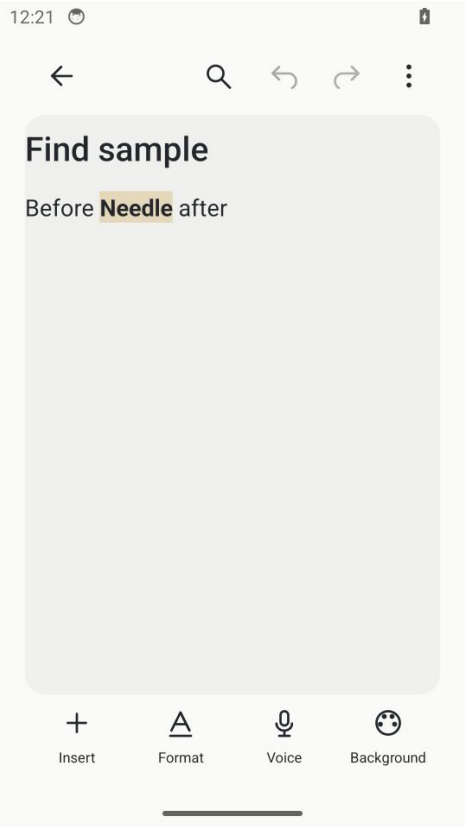
User task: Locate a phrase inside the current note and understand the hit.

Evidence: M15 displays 1 match with Writing and a short passage; M16 correctly retains a highlighted hit after returning. The useful highlight should be preserved.

Comparator: A matched Obsidian in-note-find capture is unavailable in this evidence set. O03 is vault search and is not a valid substitute.



M15 | MyNotes 0.69.0 / code 69 | note-find-sheet



M16 | MyNotes 0.69.0 / code 69 | note-find-result

Why it matters: Writing is less precise than Body text or a user-facing content label in a mixed-content note.

Direction to consider: Use concise content labels, accurate current/total hit feedback and consistent highlight visibility. Preserve the existing keyboard-aware field and successful highlighted return.

Manual acceptance: Search repeated text across body, checklist and table. Verify labels, next/previous order, close, keyboard placement and retention without changing content.

Source anchor: NotesController.showNoteFind and content match labels. Limit: Do not relabel the existing highlighted return as broken; no comparative search speed was measured.

Bug triage: distinguish defects from design concerns

Case	Current evidence and closure requirement
N01 — preview obstruction	Visible in the retained v69 home frame. Reproduce with the signed build, preserve a full-screen recording, then verify the requested floating button no longer masks readable content.
N09 — misleading empty state	Source-supported candidate. Populate a disposable library, apply a nonempty zero-match label filter and record the actual empty-state wording.
N07 — query scope broadening	Source-supported current contract. First decide and disclose the intended global/local semantics; a deliberate global search is not itself a defect.
E01 — context squeezed	Visible usability constraint. Measure readable context with the same IME/font/device before claiming a quantitative improvement.
Remaining visual/IA findings	Triage as usability work or hypotheses, not a fabricated count of crashes. Prioritise using observed frequency and severity from real tasks.
User-reported bugs	Keep the report open pending device/build, steps, expected/actual result, screen recording and opt-in diagnostics. Do not close as cannot reproduce solely because automation passed.

Safety boundary

Any reproduction uses disposable local notes with recorded originals. Do not uninstall, clear app data, replace the signed package or overwrite a user library to simplify testing. Failure injection belongs in a separate disposable environment. Keep diagnostic collection opt-in and never upload private notes or audio.

Proper hands-on comparison: preparation and controls

This is the next testing protocol, not a list of tests claimed to have run. An operator should execute and observe the same user tasks, without following developer-only shortcuts. Capture decisions, confusion, corrections and recovery, not merely a terminal pass line.

Matched setup Use the same physical Android device where possible; otherwise match OS, resolution, density, refresh rate, font scale, IME and thermal/battery conditions. Preserve signing/data safety for our installed app. Record exact builds and source evidence versions.

Matched data Create a public dummy corpus with 50 notes, five folders, three labels/tags, mixed short/long titles and repeated body phrases. Use equivalent content and user outcomes, not identical internal file formats. Add 200-item and long-note variants only for the scale tasks.

Counterbalanced order Alternate which app is used first to reduce learning effects. Separate cold and warm runs. Record novice familiarity and avoid coaching one app more than the other.

Observation Capture screen/touch recordings where consented, operator notes and available frame traces. Ask what the user expects before revealing the result. Keep errors, abandoned attempts and backtracking.

Outcome Verify that the intended note opens, edits persist, original content is retained, and navigation returns to a comprehensible state. Record unknowns instead of assigning a zero.

Hands-on task sheet A: retrieval and organisation — NOT RUN

T01 | Retrieve a remembered title

From home, find a specified note among 50 mixed items without naming its creation workspace. Verify its ID/content. Record task time, navigation switches, wrong opens and help requests.

T02 | Body-only phrase with a filter

Apply a folder/label filter, search a phrase found only in note bodies, and explain the visible scope before opening the result. Verify the expected IDs under the app’s stated contract.

T03 | Create a folder during a move

Move one dummy item into a newly named folder. Repeat Cancel, invalid name and Back branches. Verify original content, selected destination and return context.

T04 | Recover from an apparently empty list

Use a filter that returns zero items while the library remains populated. Determine why the view is empty and restore visibility without recreating notes or clearing storage.

Status: all four tasks are planned, not executed in this report. An instruction-path count would not be a measured task outcome.

Hands-on task sheet B: editing and reading — NOT RUN

T05 | Format in context

Write 250 words, select a middle phrase, apply bold and a heading, then resume typing. Record visible context, selection errors, unintended formatting, panel reopenings and saved result.

T06 | Navigate a long note

Find a named section among twenty headings and repeated words, edit it, then return to the previous location. Use each app’s best discoverable method; do not require a feature name.

T07 | Read and modify a mixed note

Read a paragraph, a checklist and a table; change one row and move one block if that operation is supported. Separate unavailable outcomes from failures. Verify unrelated content is unchanged.

T08 | Save and interruption confidence

Save, navigate away and reopen the same dummy note. In an isolated failure-injection environment, trigger a controlled save failure and observe feedback/retry. Never apply destructive testing to a real library.

Run at standard and enlarged font scales. A screenshot of a retained selection is not proof of smooth dragging or a comfortable writing session.

Smoothness, latency and effectiveness: measurement ledger

Measure	Protocol / present result
Tap to usable response	Timestamp the input and first stable usable state in recorded evidence. Separate keyboard appearance and app rendering. Present result: NOT MEASURED.
Scroll and drag quality	Capture frame traces and user-visible recordings for list scrolling, note scrolling, selection drag and reordering. State refresh rate and trace-derived jank definition. Present result: NOT MEASURED.
Cold / warm startup	Record process/cache state and time to usable content, not just launch intent dispatch. Present result: NOT MEASURED head-to-head.
Task efficiency	Time, action count, wrong turns, corrections and abandonments on the matched tasks. Present result: NOT MEASURED.
Task effectiveness	Correct item, correct scope, intended edit and saved content verified after reopen. Present result: NOT MEASURED on a matched manual task set.
Confidence / perceived simplicity	Ask users what is happening, where they are and whether work is safe; record the explanation and uncertainty. Present result: no participant study.
Resource use	Installed storage, data/cache growth, peak working memory and battery under equivalent tasks. APK payload alone is not any of these. Present result: NOT MEASURED.

Do not substitute OCR/speech benchmark timings, native test counts, JPEG file sizes or API callback success for these metrics. No claim that a fast automated call means a smooth human interaction.

Acceptance gates before calling a module polished

Readability No primary floating action masks required text in the evaluated state; no clipped essential scope/status wording; important content remains available with keyboard and large text.

Predictability Search scope and active filters are understandable; first-use empty and filtered-empty are distinct; each action’s object and return destination are clear.

Effectiveness Users reach and modify the intended content without developer assistance. Saved content matches their intent after reopen, including mixed formatting and structured blocks.

Efficiency Measured task results show fewer unnecessary transitions or less scanning; retain negative and neutral outcomes. No improvement claim before comparable measurements exist.

Smoothness Live traces/recordings show acceptable task-specific feedback and no disruptive interaction stalls under declared conditions. Define thresholds before testing, not after seeing the result.

Safety and recovery No data loss, unsafe Back navigation or silent replacement. Preserve originals and clear failure feedback; verify install-over rather than destructive reset when needed.

What to preserve; what not to copy blindly

Our plain title/body editor and labelled Insert / Format / Voice / Background actions are already understandable in M07/M08. Existing global cross-workspace search, folder/label controls, grid/list choice, highlighted find return and shared FormatControls should be improved, not described as nonexistent.

Obsidian’s useful references here are focused retrieval/action surfaces, a visible heading outline, calmer rendered prose and compact keyboard-adjacent actions. Its unlabelled icons, Markdown concepts, substantial frontmatter and distributed navigation can themselves confuse beginners. Its font-size settings also expose pixels; it is not an automatic model of plain-language typography.

Preserve our requested floating New note, single A Format entry, fifth inline Clear action, compact editor header, safe Back behaviour, direct editing, long-selection persistence and accessible per-content actions. Proposed directions are design hypotheses, not authorisation to undo earlier requirements.

No extra-feature credit is awarded for our voice/OCR/drawing tools, nor is missing a third-party plugin counted as a failure in this shared-task review. A simple app should not become a complicated clone of the comparator.

Next modules and work boundary

Stage	Scope and evidence needed
This installment	Two modules only: find/organise and edit/read. Twenty scoped findings and a proposed matched manual task set. No fixes or new Android builds.
Next: file viewers	Compare opening, navigation, selection, search, zoom, copy/export and return paths for shared file types. Use original documents and actual interactions, not only viewer launch captures.
Next: tables / lists / rich content depth	Evaluate cell/row edits, context actions, ordering and data retention as user workflows. Avoid a feature-count table that treats presence as usability.
Next: navigation / settings / recovery	Review onboarding, storage explanations, archive/trash, saving failures and recovery with disposable data. Retain signing/data safety.
Separate specialised modules	OCR, speech, drawings and other non-equivalent capabilities need their own effectiveness tests or suitable comparators, not invented Obsidian counterparts.
Implementation later	Convert validated findings into small changes, retain regression safeguards, then repeat the same manual acceptance tasks. User approval of this review is not evidence that a fix is complete.

Evidence catalogue and reproduction references

Source frames are retained in the repository; the evidence manifest stores exact paths, SHA-256 digests and dimensions. These are historical captures, not new screenshots from this review. Source images are not duplicated into this report folder.

MyNotes frames

ID	Recorded state
M01	home
M02	library
M03	folders
M04	filters-view
M05	filters-labels
M06	search-keyboard
M07	editor
M08	editor-keyboard
M09	format-keyboard
M10	format
M11	editor-folder-create
M12	notes-workspace
M13	mixed-notes-selection
M14	document-table
M15	note-find-sheet
M16	note-find-result
M17	stage-aa-selection
M18	stage-aa-typing
M19	notes-native-selection-scroll
M20	mixed-note-header

Obsidian frames

ID	Recorded state
O01	Quick switcher with file paths and Find or create input
O02	File explorer, fixture folder collapsed
O03	Vault search, empty query; not search-result quality evidence
O04	Command palette with actions
O05	Tags pane with usage counts
O06	Bold action on dedicated scratch note; keyboard visible
O07	Rendered reading view of fixture

ID	Recorded state
O08	Rendered table within note prose
O09	Heading outline pane
O10	Mobile toolbar configuration
O11	Properties in note; metadata occupies substantial space
O12	Backlinks pane
O13	Appearance font-size setting

Source/provenance references

MyNotes release qualification mynotes/docs/candidate69-markup-followup.md

MyNotes native-image provenance mynotes/docs/module-e2e/42021523354/provenance.json

MyNotes environment example
mynotes/docs/module-e2e/42021523354/light-portrait-ordered-refinement-environment-configured.json

Obsidian audit limitations mynotes/docs/comparisons/obsidian/audit/42182162974/README.md

Obsidian capture receipts mynotes/docs/comparisons/obsidian/audit/42182162974/coverage-summary.json

Obsidian source release https://github.com/obsidianmd/obsidian-releases/releases/tag/v1.15.0

Primary UI/source review mynotes/runtime/src/main/java/app/space/notes/runtime/NotesController.java

Shared formatting implementation mynotes/runtime/src/main/java/app/space/notes/runtime/FormatControls.java

Read source paths at baseline bf39d35dc625a86a84f44d03a178da4569a2b594. The current report’s findings do not silently reclassify previous app tests as manual usability evidence. No E2E success count is used as proof of polish.

Companion files: review-data.json provides structured findings; evidence-manifest.json provides source integrity; OurApp_vs_Obsidian.md provides a searchable text edition. No personal data or credentials are used.